DISPUR COLLEGE, GUWAHATI, ASSAM, INDIA

Professional Studies Department – Digital Library Documentation

Complete Technical Reference, Architecture, Cloud Setup, Usage & Deployment Guide

Professional Studies Department – Digital Library

Institution: Dispur College, Ganeshguri, Guwahati – 781006, Assam, India **Application Type:** Full-Stack Departmental Digital Library & Cloud Academic Repository **Live Deployment Status:** Supabase Cloud Backend & Storage Connected Locally (`book-covers`, `book-files`, and PostgreSQL schema provisioned) — Ready for Primary Admin Bootstrap & Vercel Hosting Deployment

Table of Contents

1. Project Overview
2. Technology Stack
3. Complete Project Folder Structure
4. Prerequisites and Software Installation
5. Environment Variables and API Keys
6. Database Setup and Configuration
7. Authentication and Admin Setup
8. Cloud File Storage Setup
9. Run the Website Locally
10. Production Deployment Guide (GitHub + Vercel + Supabase)
11. Website Usage Guide
12. API and Backend Documentation
13. Troubleshooting Guide
14. Maintenance, Backup and Updates
15. Security Best Practices

1. Project Overview

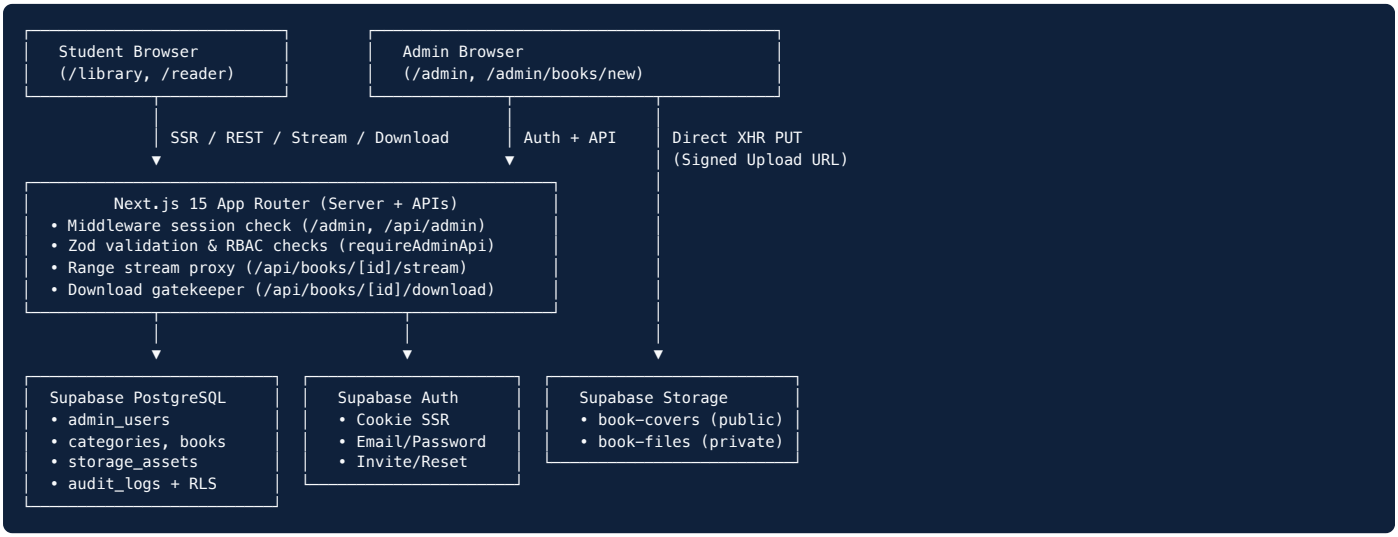
The **Professional Studies Department – Digital Library** is a centralized, cloud-backed digital repository built specifically for **Dispur College, Assam, India**. Its purpose is to give students seamless online access to departmental textbooks, lecture notes, PDFs, and EPUB e-books while giving department administrators a secure portal to publish, organize, and govern those resources.

It is intentionally designed as a streamlined departmental digital library (excluding commercial features such as payments, book issuing/returning, fines, or student registration).

Dual-Portal Architecture

1. **Student Portal (`/library`)** — Open to students without requiring login:
   - Homepage (`/library`, with `/` redirecting to `/library`) displaying departmental branding, live book/category counters, search bar, featured titles, recently added books, and browse-by-category cards.
   - Full searchable and filterable catalog (`/library/browse`) supporting search by title, author, subject, publisher, description, and category; filtering by category, subject, file format (`PDF` / `EPUB`), and downloadability; sorting; and pagination.
   - Book Details page (`/library/books/[id]`) displaying publication metadata, cover art (or academic typographic cover fallback), online reading link, conditional download button, and related books.
   - Integrated Browser Reader (`/library/books/[id]/read`) powered by **PDF.js** for `.pdf` files and **EPUB.js** for `.epub` files.
2. **Admin Portal (`/admin`)** — Protected administrative control panel (no public registration):
   - Secure sign-in (`/admin/login`) and password update/invitation completion (`/admin/update-password`).
   - Overview dashboard (`/admin`) showing book/format/category counts, cloud storage usage, recently uploaded books, and admin audit logs.
   - Book management (`/admin/books`, `/admin/books/new`, `/admin/books/[id]/edit`) with direct-to-cloud signed file uploads, upload progress bars, file replacement, download permission toggles, and synchronized storage deletion.
   - Category management (`/admin/categories`) and multi-administrator team management (`/admin/team`).

How System Components Communicate



1. **Direct Signed Uploads:** When an administrator selects a PDF/EPUB or cover image, the browser calls `POST /api/admin/uploads/sign`. The Next.js backend verifies the administrator's session and validates the file's extension, MIME type, and size, then asks Supabase Storage for a one-time signed upload URL. The browser uploads the file directly to Supabase Storage via `XMLHttpRequest` (`PUT` with `FormData`) so large textbooks bypass serverless body limits and report real byte-level progress (`0%–100%`). Afterward, `POST /api/admin/books` records the book and `storage_assets` metadata in PostgreSQL.
2. **Private Document Streaming & Downloads:** PDF and EPUB documents reside in a **private** Supabase Storage bucket (`book-files`). When a student opens the online reader, PDF.js/EPUB.js requests `/api/books/[id]/stream`, which fetches a short-lived signed URL on the server and proxies HTTP `Range` byte chunks (`206 Partial Content`) with `Content-Disposition: inline`. Downloads route through `/api/books/[id]/download`, which checks `books.allow_download` in PostgreSQL before redirecting to a 60-second signed attachment URL (or returning `403 Forbidden` when disabled).

---

2. Technology Stack

Layer	Technology / Package	Version	Why It Is Used
Framework	Next.js (`next`)	`^15.3.0`	App Router for SSR pages, Server Components, Middleware, and secure backend API Route Handlers in one maintainable deployment.
UI Library	React (`react`, `react-dom`)	`^19.0.0`	Component-based interactive UI across Student and Admin portals.
Language	TypeScript (`typescript`)	`^5.8.2`	Strict compile-time type safety across database models, API payloads, and UI props.
Styling	Tailwind CSS (`tailwindcss`, `postcss`, `autoprefixer`)	`^3.4.17`	Responsive utility-first styling configured with the college palette (Deep Navy, Warm White/Ivory, Soft Beige/Parchment, Muted Gold).
Typography	`@fontsource/merriweather`, `@fontsource/plus-jakarta-sans`, `@fontsource/jetbrains-mono`	`^5.2.5`	Self-hosted academic serif, UI sans-serif, and monospace fonts bundled offline with zero external Google Fonts runtime dependency.
Icons	Lucide React (`lucide-react`)	`^0.487.0`	Clean academic and administrative UI icons.
Database & Auth SDK	`@supabase/supabase-js` & `@supabase/ssr`	`^2.49.4` / `^0.6.1`	Connects Next.js Server Components, Middleware, API routes, and browser clients to Supabase PostgreSQL, Auth (cookie-based SSR sessions), and Cloud Storage.
Node.js Transport & Migrations	`ws` & `pg`	`^8.x`	Provides WebSocket transport for Supabase SDK on Node.js 20 (`src/lib/supabase/realtime.ts`) and direct PostgreSQL execution in `scripts/migrate.mjs`.
PDF Viewer	PDF.js (`pdfjs-dist`)	`^4.10.38`	Renders PDFs directly onto HTML `<canvas>` inside the browser with zoom, fit-width, page jump, and byte-range streaming using local worker `public/pdf.worker.min.mjs`.
EPUB Reader	`epubjs` + `jszip`	`^0.3.93` / `^3.10.1`	Parses and renders `.epub` e-books directly in the browser with Table of Contents navigation, font resizing, and theme modes.
Validation	Zod (`zod`)	`^3.24.2`	Schema validation for book creation/updates, categories, administrator accounts, UUID params, and signed upload requests.
Testing	Vitest (`vitest`)	`^3.1.1`	Fast unit and route-security test runner verifying validation, environment checks, RBAC rules, and download/auth enforcement.
Linting	ESLint (`eslint`, `eslint-config-next`)	`^9.23.0`	Static code quality and Next.js rule enforcement.

Layer	Technology / Package	Version	Why It Is Used
Cloud Backend	Supabase	Cloud	Managed PostgreSQL database with RLS, authentication, and S3-backed object storage buckets.
Hosting	Vercel	Cloud	Zero-config Next.js production hosting configured via `vercel.json`.

3. Complete Project Folder Structure

```
.
├── .env.example                                # Safe environment variable template
├── .gitignore                                  # Ignores .env.local, node_modules, .next, generated worker
├── README.md                                   # Project documentation
├── eslint.config.mjs                           # ESLint configuration extending Next.js core-web-vitals & TypeScript
├── next.config.ts                              # Next.js config: Supabase image domains, canvas alias, security headers
├── next-env.d.ts                               # Auto-generated Next.js TypeScript declarations
├── package.json                                # Dependencies and npm scripts
├── package-lock.json                           # Locked npm dependency tree
├── postcss.config.mjs                          # PostCSS config for Tailwind CSS & Autoprefixer
├── tailwind.config.ts                          # Academic theme palette, fonts, shadows, and content paths
├── tsconfig.json                               # Strict TypeScript compiler configuration (@/* -> ./src/*)
├── vercel.json                                 # Vercel deployment settings & API streaming/download timeouts
├── vitest.config.ts                            # Vitest test runner configuration
│
├── public/
│   └── pdf.worker.min.mjs                      # Copied automatically on postinstall/build by copy-pdf-worker.mjs
│
├── scripts/
│   ├── bootstrap-admin.mjs                     # CLI script to create the initial super_admin in Auth + admin_users
│   ├── copy-pdf-worker.mjs                     # Copies pdfjs-dist worker into public/pdf.worker.min.mjs
│   ├── migrate.mjs                             # Executes initial SQL schema migration & creates storage buckets
│   ├── verify-setup.mjs                        # Verifies Supabase tables, storage buckets, and env connectivity
│   └── lib/
│       └── env-loader.mjs                      # Shared .env.local parser for CLI scripts
│
├── supabase/
│   └── migrations/
│       └── 202611100001_initial_schema.sql     # Complete Postgres schema, indexes, RLS policies, RPC & Storage buckets
│
├── tests/
│   ├── env.test.ts                             # Tests placeholder detection and key fallback logic
│   ├── security-routes.test.ts                 # Tests API auth, RBAC, download enforcement & storage cleanup (mocked Supabase)
│   └── validation.test.ts                      # Tests Zod schemas, file validation, path traversal & search sanitization
│
└── src/
    ├── middleware.ts                           # Edge middleware refreshing SSR cookies & guarding /admin and /api/admin
    │
    ├── types/
    │   └── library.ts                          # Shared TypeScript interfaces for books, categories, admins, assets & logs
    │
    ├── lib/
    │   ├── env.ts                              # Safe env accessors, placeholder checks & upload size limit helpers
    │   ├── validation.ts                       # Zod schemas, UUID/path/MIME validation, formatting & RBAC helpers
    │   ├── auth/
    │   │   └── admin.ts                        # Server helpers: getCurrentAdmin(), requireAdminApi(), recordAuditLog()
    │   ├── queries/
    │   │   ├── admin.ts                        # Admin portal DB queries (overview KPIs, books, categories, team)
    │   │   ├── library.ts                      # Student portal DB queries (home data, search/filter/pagination, book detail)
    │   │   └── mappers.ts                      # Shared RawBookRow -> BookWithCategory + cover URL mapper
    │   └── supabase/
    │       ├── admin.ts                        # Server-only privileged Supabase client using SUPABASE_SECRET_KEY
    │       ├── browser.ts                      # Singleton Supabase SSR browser client for Client Components
    │       ├── realtime.ts                     # Node.js 20 WebSocket transport helper for server Supabase clients
    │       ├── server.ts                       # Cookie-aware SSR server client & anonymous public catalog client
    │       └── storage.ts                      # Bucket constants, UUID storage path builder, public cover URL & safe cleanup
    │
    ├── components/
    │   ├── admin/
    │   │   ├── AdminBooksManager.tsx           # Admin books table with search, filters, inline toggles & delete modal
    │   │   ├── AdminLoginForm.tsx              # Email/password sign-in & neutral password-reset request form
    │   │   ├── AdminShell.tsx                  # Responsive admin sidebar, header, role badge, password link & logout
    │   │   ├── BookForm.tsx                    # Create/Edit book form with direct signed cloud uploads & progress bars
    │   │   ├── CategoriesManager.tsx           # Category CRUD grid & create/edit modal with icon picker
    │   │   └── TeamManager.tsx                 # Multi-admin directory, onboarding modal (password/invite) & role controls
    │   ├── library/
    │   │   ├── BookCard.tsx                    # Student book card with cover, badges, synopsis & Read/Download actions
    │   │   ├── CatalogFilterBar.tsx            # Search bar + category, subject, format, permission & sort controls
    │   │   ├── LibraryFooter.tsx               # Institutional footer for Dispur College
    │   │   ├── LibraryNavbar.tsx               # Responsive student navigation bar with mobile menu & Admin Portal link
    │   │   └── SearchBar.tsx                   # Homepage hero search bar redirecting to /library/browse
    │   ├── reader/
    │   │   ├── BookReaderShell.tsx             # Records view metric and loads PdfReader or EpubReader by file_type
    │   │   ├── EpubReader.tsx                  # Interactive EPUB.js reader with TOC drawer, font size, themes & fullscreen
    │   │   └── PdfReader.tsx                   # Interactive PDF.js canvas reader with zoom, fit-width, paging & fullscreen
    │   └── shared/
    │       ├── BookCover.tsx                   # Renders cover image with fallback academic hardbound spine artwork
    │       ├── CategoryIcon.tsx                # Maps icon_name strings to Lucide React icons
    │       └── SetupNotice.tsx                 # Displayed cleanly when Supabase env variables are not configured yet
    │
    └── app/
        ├── globals.css                         # Tailwind directives, font/color utilities, button/input classes & scrollbars
        ├── icon.svg                            # College Digital Library SVG emblem / favicon
        ├── layout.tsx                          # Root HTML layout importing bundled @fontsource fonts & metadata
        ├── not-found.tsx                       # Custom academic 404 page
        ├── page.tsx                            # Root route (/) redirecting cleanly to /library
        │
        ├── auth/
        │   └── callback/
        │       └── route.ts                    # PKCE ?code= session exchange endpoint redirecting to /admin/update-password
        │
        ├── library/                            # Student Portal Routes
        │   ├── page.tsx                        # Homepage (/library)
        │   ├── browse/
```

```
│   │   └── page.tsx                    # Filterable, sortable, paginated catalog (/library/browse)
│   └── books/
│       └── [id]/
│           ├── page.tsx                # Book Details page (/library/books/[id])
│           └── read/
│               └── page.tsx            # Online PDF/EPUB Reader page (/library/books/[id]/read)
│
├── admin/                              # Admin Portal Routes
│   ├── login/
│   │   └── page.tsx                    # Administrator login page (/admin/login)
│   ├── update-password/
│   │   └── page.tsx                    # Password setup for invited/resetting/logged-in admins
│   └── (protected)/                    # Server-guarded admin layout group
│       ├── layout.tsx                  # Verifies active admin session via getCurrentAdmin() + renders AdminShell
│       ├── page.tsx                    # Admin Dashboard (/admin)
│       ├── categories/
│       │   └── page.tsx                # Manage Categories (/admin/categories)
│       ├── team/
│       │   └── page.tsx                # Manage Administrators (/admin/team)
│       └── books/
│           ├── page.tsx                # Manage Books (/admin/books)
│           ├── new/
│           │   └── page.tsx            # Upload New Book (/admin/books/new)
│           └── [id]/
│               └── edit/
│                   └── page.tsx        # Edit Book & Replace Files (/admin/books/[id]/edit)
│
└── api/                                # Backend API Route Handlers
    ├── health/route.ts                 # Service & Supabase config status check
    ├── auth/logout/route.ts            # Signs out active Supabase Auth session
    ├── categories/route.ts             # Public category list with book counts
    ├── books/
    │   ├── route.ts                    # Public catalog search/filter/sort/pagination API
    │   └── [id]/
    │       ├── route.ts                # Public single book details API
    │       ├── stream/route.ts         # Byte-range proxy streaming PDFs/EPUBs from private book-files bucket
    │       ├── download/route.ts       # Download gatekeeper checking allow_download before signed URL redirect
    │       └── view/route.ts           # Records view count metric via increment_book_metric RPC
    └── admin/
        ├── overview/route.ts           # Admin dashboard KPIs, recent books & audit logs
        ├── uploads/
        │   ├── sign/route.ts           # Validates file metadata & issues signed Supabase upload URL
        │   └── cleanup/route.ts        # Removes uncommitted uploaded storage objects safely
        ├── books/
        │   ├── route.ts                # List admin books (GET) & create book record (POST)
        │   └── [id]/route.ts           # Get (GET), update/replace files (PATCH), delete book+files (DELETE)
        ├── categories/
        │   ├── route.ts                # List (GET) & create category (POST)
        │   └── [id]/route.ts           # Update (PATCH) & delete category (DELETE, guarded if books exist)
        └── team/
            ├── route.ts                # List admins (GET) & create/invite admin (POST, super_admin only)
            └── [id]/route.ts           # Update role/status (PATCH) & delete admin (DELETE, super_admin only)
```

4. Prerequisites and Software Installation

Install and prepare the following tools before setting up the project:

1. **Node.js (`v20.18.0` or newer LTS recommended)** — Includes `npm`.
   - Download: https://nodejs.org/
   - Verify after installation:

     ```
     node –v
     npm –v
     ```

2. **Package Manager (`npm`)** — Bundled with Node.js (tested with `npm 10.8.2`, using `package-lock.json`).
3. **Git** — For version control and pushing code to GitHub.
   - Download: https://git-scm.com/
   - Verify:

     ```
     git ––version
     ```

4. **Code Editor** — Visual Studio Code (recommended).
5. **Supabase Account** — Free or Pro plan at https://supabase.com/ for PostgreSQL, Auth, and Storage.
6. **Vercel Account** — Free Hobby or Pro plan at https://vercel.com/ (or any Node.js hosting provider) for production deployment.
7. **GitHub Account** — https://github.com/ to host the repository and connect automated Vercel deployments.

5. Environment Variables and API Keys

Environment Variable Reference

The application reads configuration in `src/lib/env.ts` from the variables listed in `.env.example`:

Variable Name	Purpose	Where to Obtain	Visibility	Where to Configure
`NEXT_PUBLIC_SUPABASE_URL`	Base URL of your Supabase project (`https://<ref>.supabase.co`).	Supabase Dashboard → **Project Settings** → **Data API** / **API Keys**	Public (Safe in browser)	`.env.local` & Vercel
`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`	Public client key used by browser/SSR auth with Row Level Security. *(Also supports legacy `NEXT_PUBLIC_SUPABASE_ANON_KEY` as fallback.)*	Supabase Dashboard → **Project Settings** → **API Keys** → **Publishable key** (`sb_publishable_...`)	Public (Protected by RLS)	`.env.local` & Vercel
`SUPABASE_SECRET_KEY`	Privileged server-only key bypassing RLS for signed upload/download URLs, streaming private files, deleting storage files, and managing admins. *(Also supports legacy `SUPABASE_SERVICE_ROLE_KEY` as fallback.)*	Supabase Dashboard → **Project Settings** → **API Keys** → **Secret key** (`sb_secret_...`)	**SECRET (Server-Only)**	`.env.local` & Vercel (Never prefix with `NEXT_PUBLIC_`)
`DATABASE_URL`	Optional PostgreSQL URI used **only** if running external CLI migrations/ `pg_dump` backups; **not** used by Next.js at runtime.	Supabase Dashboard → **Project Settings** → **Database** → **Connection string (URI)**	**SECRET (CLI/Backup Only)**	Optional in `.env.local`
`NEXT_PUBLIC_SITE_URL`	Canonical public URL (`http://localhost:3000` locally or `https://your-domain.vercel.app` in production) used for invite redirect links.	Your local address or deployed domain	Public	`.env.local` & Vercel
`MAX_BOOK_FILE_SIZE_MB`	Maximum book document upload size in MB (defaults to `50` if omitted).	Chosen by department admin (must be ≤ Supabase bucket limit `100` MB)	Server & passed as prop	Optional in `.env.local` & Vercel
`MAX_COVER_FILE_SIZE_MB`	Maximum book cover image upload size in MB (defaults to `5` if omitted).	Chosen by department admin (must be ≤ `5` MB bucket limit)	Server & passed as prop	Optional in `.env.local` & Vercel
`CLOUDFLARE_ACCOUNT_ID` , `CLOUDFLARE_ACCESS_KEY_ID` , `CLOUDFLARE_SECRET_ACCESS_KEY` , `CLOUDFLARE_BUCKET_NAME`	**Not currently active in application code.** Commented placeholders in `.env.example` for optional future Cloudflare R2 migration; Supabase Storage is the active storage provider.	Cloudflare Dashboard → R2 *(only if integrated later)*	**SECRET**	Leave unset/commented

> **Security Warning:**
>
> - **NEVER** prefix `SUPABASE_SECRET_KEY` (or `SUPABASE_SERVICE_ROLE_KEY` / `DATABASE_URL`) with `NEXT_PUBLIC_` . Any `NEXT_PUBLIC_*` variable is embedded into browser JavaScript bundles.
> - **NEVER** commit `.env` , `.env.local` , or `.env.*.local` to GitHub. `.gitignore` already excludes them.
> - **NEVER** hardcode real credentials into source files or README files.

Step-by-Step: Creating a Supabase Project & Obtaining Keys

1. Sign in at https://supabase.com/dashboard and click **New project**.
2. Choose your Organization, enter a project name (e.g., `dispur-college-digital-library`), generate a strong database password (save it securely), select the **South Asia (Mumbai)** region (`ap-south-1`) for lowest latency in Assam/India, and click **Create new project**.
3. Wait ~1–2 minutes for provisioning to finish.
4. Go to **Project Settings** (gear icon in the left sidebar) → **API Keys** (or **Data API**):
   - Copy the **Project URL** → `NEXT_PUBLIC_SUPABASE_URL` .
   - Copy the **Publishable key** (`sb_publishable_...`) → `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` *(if your project uses legacy JWT keys, copy `anon public` into `NEXT_PUBLIC_SUPABASE_ANON_KEY`)*.
   - Reveal and copy the **Secret key** (`sb_secret_...`) → `SUPABASE_SECRET_KEY` *(or `service_role` into `SUPABASE_SERVICE_ROLE_KEY`)*.

Adding Environment Variables Locally

1. In the project root, copy the template:

```
cp .env.example .env.local
```

2. Open `.env.local` and replace the placeholder values with your real Supabase values:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-actual-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_your_actual_key
SUPABASE_SECRET_KEY=sb_secret_your_actual_secret_key
NEXT_PUBLIC_SITE_URL=http://localhost:3000
MAX_BOOK_FILE_SIZE_MB=50
MAX_COVER_FILE_SIZE_MB=5
```

(Leave `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, and `SUPABASE_SECRET_KEY` out of source control. Until configured, `/api/health` reports `supabaseConfigured: false` and the UI displays a configuration notice.)

Adding Environment Variables in Vercel

In **Vercel Dashboard → Your Project → Settings → Environment Variables**, add `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, `SUPABASE_SECRET_KEY`, `NEXT_PUBLIC_SITE_URL` (your production `https://...` URL), and optionally `MAX_BOOK_FILE_SIZE_MB` and `MAX_COVER_FILE_SIZE_MB`, then select **Production**, **Preview**, and **Development** environments and save.

---

6. Database Setup and Configuration

The database schema is defined in `supabase/migrations/202611100001_initial_schema.sql`. **It intentionally starts completely empty**—no demo users, sample books, or pre-filled categories are inserted.

Database Tables and Relationships

Table	Purpose	Key Columns & Constraints	Relationships
`public.admin_users`	Tracks authorized departmental administrators and roles.	`id` (PK), `name`, `email` (UNIQUE), `role` (`super_admin` | `admin` | `editor`), `is_active`, `invited_by`, `created_at`, `updated_at`	`id` → `auth.users(id)` (`ON DELETE CASCADE`); `invited_by` → `admin_users(id)` (`ON DELETE SET NULL`)
`public.categories`	Departmental subject categories for organizing books.	`id` (PK), `category_name` (UNIQUE), `slug` (UNIQUE), `description`, `icon_name`, `created_at`, `updated_at`	Referenced by `books.category_id` (`ON DELETE RESTRICT` so categories with books cannot be orphaned)
`public.books`	Central catalog of digital books and PDF/EPUB metadata.	`id` (PK), `title`, `author`, `description`, `subject`, `category_id`, `publisher`, `publication_year` (`1400..2100`), `isbn`, `language`, `page_count`, `cover_image_path`, `file_storage_path`, `file_name`, `file_type` (`pdf` | `epub`), `mime_type`, `file_size` (`> 0`), `allow_download`, `is_featured`, `view_count`, `download_count`, `uploaded_by`, `created_at`, `updated_at`	`category_id` → `categories(id)` (`ON DELETE RESTRICT`); `uploaded_by` → `admin_users(id)` (`ON DELETE SET NULL`)
`public.storage_assets`	Synchronized registry of every uploaded book file and cover image in cloud storage.	`id` (PK), `book_id`, `asset_kind` (`book_file` | `cover_image`), `bucket_name` (`book-files` | `book-covers`), `storage_path` (UNIQUE), `original_filename`, `mime_type`, `size_bytes`, `uploaded_by`, `created_at`	`book_id` → `books(id)` (`ON DELETE CASCADE`); `uploaded_by` → `admin_users(id)` (`ON DELETE SET NULL`)
`public.audit_logs`	Accountability log of admin actions (`book.created`, `book.updated`, `book.deleted`, `category.*`, `admin.*`).	`id` (PK), `actor_id`, `actor_email`, `action`, `target_type`, `target_id`, `details` (`jsonb`), `created_at`	`actor_id` → `admin_users(id)` (`ON DELETE SET NULL`)

Indexes, Triggers, Functions & Row Level Security (RLS)

- **Extensions:** Enables `pgcrypto` and `pg_trgm`.
- **Indexes:** B-tree indexes on `books(category_id)`, `books(created_at DESC)`, `books(is_featured)`, `books(file_type)`, `books(subject)`, `storage_assets(book_id)`, `audit_logs(created_at DESC)`, plus GIN trigram indexes (`gin_trgm_ops`) on `title`, `author`, `subject` and a full-text GIN `tsvector` index (`idx_books_fts`).
- **Triggers:** `set_updated_at()` keeps `updated_at` current on `admin_users`, `categories`, and `books`.
- **Security Functions:**
  - `public.is_active_admin()` and `public.has_admin_role(allowed_roles text[])`: `SECURITY DEFINER` (`SET search_path = public`) functions reading `auth.uid()` directly (execution revoked from `PUBLIC`/`anon`, granted only to `authenticated` and `service_role`) to evaluate admin RLS policies without recursive table lookups.
  - `public.increment_book_metric(p_book_id uuid, p_metric text)`: `SECURITY DEFINER` RPC allowing public students to increment `view_count` or `download_count` (only when `allow_download = true`) without granting `UPDATE` privileges on `books`.
- **RLS Policies Summary:**
  - `categories` & `books`: Public `SELECT` (`anon`, `authenticated`); `INSERT`/`UPDATE` restricted to active admins; `DELETE` restricted to `super_admin` and `admin`.

- `admin_users`: `SELECT` restricted to active admins; `INSERT` / `UPDATE` / `DELETE` restricted to `super_admin` (`DELETE` also forbids deleting `auth.uid()`).
- `storage_assets` & `audit_logs`: Accessible only to active admins (`storage_assets` delete restricted to `super_admin` and `admin`).

Executing the SQL Migration

You can apply `supabase/migrations/202611100001_initial_schema.sql` either via CLI (`npm run db:migrate`) or directly inside the Supabase SQL Editor:

1. **CLI Method (using `DATABASE_URL` in `.env.local`):**

```
npm run db:migrate
```

2. **Supabase Dashboard SQL Editor Method:**
   - Open **Supabase Dashboard → SQL Editor → New query**, paste the contents of `supabase/migrations/202611100001_initial_schema.sql`, and click **Run**.
3. Verify your tables, buckets, and connection locally by running:

```
npm run verify-setup
```

Expected output on a clean new database:

```
Checking Supabase Database Tables...
  [OK]   Table "admin_users" (0 rows)
  [OK]   Table "categories" (0 rows)
  [OK]   Table "books" (0 rows)
  [OK]   Table "storage_assets" (0 rows)
  [OK]   Table "audit_logs" (0 rows)
Checking Supabase Cloud Storage Buckets...
  [OK]   Bucket "book-covers" (public)
  [OK]   Bucket "book-files" (private)

[NEXT STEP] Database is clean and empty. Run "npm run bootstrap-admin" to create your first Primary Administrator.
```

---

7. Authentication and Admin Setup

How Admin Authentication Works

- Authentication uses **Supabase Auth** (email + password, secure password hashing handled by Supabase) with HTTP-only SSR cookies via `@supabase/ssr`.
- Public signup/registration is **completely disabled**.
- Three defense layers enforce authorization on every admin page and API request:
  1. **Edge Middleware (`src/middleware.ts`):** Refreshes auth cookies on `/admin/*` and `/api/admin/*`. Unauthenticated requests receive `401` on `/api/admin/*` or redirect to `/admin/login` on `/admin/*` (excluding `/admin/login`).
  2. **Protected Server Layout (`src/app/admin/(protected)/layout.tsx`):** Calls `getCurrentAdmin()` to verify that the authenticated Supabase user exists in `public.admin_users` with `is_active = true`. Non-admin or suspended accounts are redirected to `/admin/login?error=unauthorized`.
  3. **API Authorization Guard (`requireAdminApi()`):** Every admin route handler verifies `supabase.auth.getUser()`, verifies an active `admin_users` record, and enforces role permissions (`super_admin`, `admin`, `editor`).

Creating the First Administrator Securely

After configuring `.env.local` and running the SQL migration, create your first `super_admin` using `scripts/bootstrap-admin.mjs`:

```
npm run bootstrap-admin
```

You will be prompted interactively for:

- **Primary Admin Full Name** (minimum 2 characters)
- **Primary Admin Email**
- **Primary Admin Password** (minimum 8 characters)

*(You may also pass `--name`, `--email`, and `--password` CLI flags, or create the user in **Supabase Dashboard → Authentication → Users** and insert their `id`, `name`, `email`, and `'super_admin'` role into `public.admin_users` via SQL.)*

Roles and Permissions Matrix

Capability	`super_admin` (Primary Admin)	`admin` (Department Admin)	`editor` (Library Curator)
View Admin Dashboard & Audit Logs	Yes	Yes	Yes
Upload Books, Replace Files & Edit Metadata	Yes	Yes	Yes

Capability	`super_admin` (Primary Admin)	`admin` (Department Admin)	`editor` (Library Curator)
Toggle Student Download / Featured Flags	Yes	Yes	Yes
Create & Edit Categories	Yes	Yes	Yes
Delete Books & Cloud Storage Files	Yes	Yes	**No (`403`)**
Delete Unused Categories	Yes	Yes	**No (`403`)**
View Administrator Directory	Yes	Yes	Yes
Add, Change Role, Suspend, or Remove Admins	Yes	**No (`403`)**	**No (`403`)**

Authorizing Additional Administrators

1. Sign in as the `super_admin` at `/admin/login` and open **Administrators** (`/admin/team`).
2. Click **Add Administrator**.
3. Enter the faculty member's Full Name, Email, and Role (`admin` or `editor`), and select an onboarding method:
   - **Set Temporary Password:** Creates the confirmed Supabase Auth user immediately (`email_confirm: true`) so they can sign in right away (works even without custom SMTP configured). They should then change their password via **Change Password** in the sidebar.
   - **Send Email Invite:** Calls `inviteUserByEmail` and emails a link pointing to `/admin/update-password` where the invited admin chooses their own password *(requires Supabase Auth email delivery / custom SMTP and matching Redirect URLs)*.

Managing Members & Resetting Passwords

- **Suspend / Reactivate / Role Change / Remove:** From `/admin/team` , `super_admin` can suspend an account (`is_active: false` , blocking login immediately), switch roles between `admin` and `editor` , or delete an administrator (which deletes both their `admin_users` row and their `auth.users` account). Self-modification and demoting/deleting `super_admin` accounts are blocked.
- **Change Password (Signed-In):** Click **Change Password** in the admin sidebar (`/admin/update-password`).
- **Forgot Password:** On `/admin/login` , click **Forgot password?**, enter your administrator email, and follow the email reset link via `/auth/callback?next=/admin/update-password` .

Troubleshooting Login Issues

- **"Your account does not have active administrator privileges":** The user exists in Supabase `auth.users` but either has no matching row in `public.admin_users` or `is_active` is `false` . Verify their row in `public.admin_users` or re-run `npm run bootstrap-admin` for the primary admin.
- **"Supabase environment variables are not configured yet":** `.env.local` is missing or still contains placeholder strings.
- **Invite / Reset Email Fails:** Supabase's built-in emailer has strict hourly limits and only emails team members unless custom SMTP is configured. Use **Set Temporary Password** in `/admin/team` or configure custom SMTP in **Supabase Dashboard → Project Settings → Auth**.

---

8. Cloud File Storage Setup

All uploaded covers, PDFs, and EPUBs are stored exclusively in **Supabase Storage** (never saved permanently on the web server filesystem).

Storage Buckets & Access Policies

Created automatically by `supabase/migrations/202611100001_initial_schema.sql` :

Bucket ID	Visibility	File Size Ceiling	Allowed MIME Types	Access Rules
`book-covers`	**Public**	`5 MB` (`5242880` bytes)	`image/jpeg` , `image/png` , `image/webp`	Public `SELECT` for fast catalog thumbnails; active admins can `INSERT` / `UPDATE` ; `super_admin` / `admin` can `DELETE` .
`book-files`	**Private**	`100 MB` (`104857600` bytes; app defaults to `50 MB` via `MAX_BOOK_FILE_SIZE_MB`)	`application/pdf` , `application/epub+zip`	**No public access.** Only active admins can `SELECT` / `INSERT` / `UPDATE` directly via RLS; `super_admin` / `admin` can `DELETE` . Students read/download only via backend API routes.

Upload, Replace, Delete, Read & Download Flows

- **Format & Size Limits:**
  - Book documents: `.pdf` (`application/pdf`) and `.epub` (`application/epub+zip`), up to `MAX_BOOK_FILE_SIZE_MB` (default `50 MB` , configurable up to the `100 MB` bucket limit).
  - Cover images: `.jpg` , `.jpeg` , `.png` , `.webp` , up to `MAX_COVER_FILE_SIZE_MB` (default `5 MB`).
  - Storage paths are sanitized via `sanitizeStorageFilename()` and prefixed with `books/<year>/<uuid>-...` or `covers/<year>/<uuid>-...` .
- **Upload & Replace:** `POST /api/admin/uploads/sign` issues a signed upload URL. On creation (`POST /api/admin/books`) or replacement (`PATCH /api/admin/books/[id]`), `storage_assets` is updated and any replaced previous file is deleted from Supabase Storage. If a user uploads a file and clears/replaces it before saving, `POST /api/admin/uploads/cleanup` removes the unattached file.
- **Delete Synchronization:** `DELETE /api/admin/books/[id]` removes the `books` row (cascading `storage_assets`) and deletes the book file from `book-files` and cover image from `book-covers` .

- **Online Reading:** `GET /api/books/[id]/stream` streams byte ranges (`Accept-Ranges: bytes`, `206 Partial Content`, `Content-Disposition: inline`, `Cache-Control: private, no-store`) to PDF.js / EPUB.js.
- **Secure Downloads:** `GET /api/books/[id]/download` verifies `allow_download === true`, increments `download_count`, and issues a `302` redirect to a 60-second signed Supabase Storage download URL (`download: cleanFileName`). If `allow_download === false`, it returns `403 Forbidden`.

---

9. Run the Website Locally

1. **Navigate to the project directory:**

```
cd "/path/to/your/project"
```

2. **Install dependencies:**

```
npm install
```

(Automatically executes `postinstall`, running `scripts/copy-pdf-worker.mjs` to copy `pdf.worker.min.mjs` into `public/`.)

3. **Configure environment variables:**

```
cp .env.example .env.local
```

Fill in your Supabase credentials inside `.env.local` (see Section 5).

4. **Apply the database & storage migration:**

Run `npm run db:migrate` (or execute `supabase/migrations/202611100001_initial_schema.sql` in the **Supabase SQL Editor**), then verify:

```
npm run verify-setup
```

5. **Create the Primary Administrator account:**

```
npm run bootstrap-admin
```

6. **Start the development server:**

```
npm run dev
```

7. **Open in browser:**
   - Student Portal: http://localhost:3000/library
   - Admin Portal: http://localhost:3000/admin/login
8. **Run quality checks & local production build:**

```
npm run typecheck
npm run lint
npm test
npm run build
npm run start
```

---

10. Production Deployment Guide (GitHub + Vercel + Supabase)

Follow these steps to deploy the Digital Library live online:

1. **Create a GitHub Repository:**
   - Go to https://github.com/new and create a **Private** (or Public) empty repository (do not initialize with README/gitignore since the project already has them).
2. **Push the Project to GitHub:**

Verify `.env.local` is **not** tracked (`git status` should never list `.env.local`), then run:

```
git init
git add .
git commit -m "Initial production release: Dispur College Digital Library"
git branch -M main
git remote add origin https://github.com/<your-username>/<your-repo-name>.git
git push -u origin main
```

3. **Import into Vercel:**
   - Log in at https://vercel.com/new, select **Import Git Repository**, and choose your GitHub repository.
4. **Configure Build Settings:**
   - Vercel automatically detects **Next.js** (also specified in `vercel.json`):
     - **Framework Preset:** `Next.js`

- **Build Command:** `npm run build` *(runs `node scripts/copy-pdf-worker.mjs && next build`)*
- **Install Command:** `npm install`
- **Output Directory:** `.next` (default)

5. **Add All Required Environment Variables in Vercel:**

   Under **Settings → Environment Variables**, add:

   - `NEXT_PUBLIC_SUPABASE_URL`
   - `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` *(or `NEXT_PUBLIC_SUPABASE_ANON_KEY`)*
   - `SUPABASE_SECRET_KEY` *(or `SUPABASE_SERVICE_ROLE_KEY`)*
   - `NEXT_PUBLIC_SITE_URL` *(set to your Vercel URL, e.g., `https://dispur-digital-library.vercel.app`)*
   - `MAX_BOOK_FILE_SIZE_MB` (`50`)
   - `MAX_COVER_FILE_SIZE_MB` (`5`)

6. **Ensure Supabase Production Database & Storage Are Configured:**
   - Confirm `supabase/migrations/202611100001_initial_schema.sql` has been executed on your Supabase project.
   - Confirm `book-covers` (public) and `book-files` (private) buckets exist under **Supabase Dashboard → Storage**.
7. **Configure Supabase Auth Redirect URLs:**
   - Go to **Supabase Dashboard → Authentication → URL Configuration**.
   - Set **Site URL** to your production domain (`https://dispur-digital-library.vercel.app`).
   - Under **Redirect URLs**, add:
     - `https://dispur-digital-library.vercel.app/**`
     - `http://localhost:3000/**` *(if developing locally as well)*
8. **Deploy:**
   - Click **Deploy** in Vercel and wait for the build to succeed.
9. **Bootstrap Your Production Primary Admin (if not already done locally against the same Supabase project):**
   - Run `npm run bootstrap-admin` locally pointing `.env.local` at your production Supabase project, or create the user in **Supabase Dashboard → Authentication → Users** and insert into `public.admin_users`.
10. **Connect a Custom Domain (Optional):**
    - In **Vercel → Project Settings → Domains**, add your college subdomain (e.g., `library.dispurcollege.ac.in`), update your DNS `CNAME` / `A` records as instructed by Vercel, and update `NEXT_PUBLIC_SITE_URL` + Supabase **Site URL** / **Redirect URLs** to match the custom domain.
11. **Post-Deployment Verification Checklist:**
    - ☐ Visit `https://<your-domain>/api/health` → confirms `"status": "ok"`, `"supabaseConfigured": true`, `"supabaseAdminConfigured": true`.
    - ☐ Visit `https://<your-domain>/library` → loads Student Portal homepage cleanly.
    - ☐ Visit `https://<your-domain>/admin/login` → sign in with your Primary Admin account.
    - ☐ Create at least one category under `/admin/categories`.
    - ☐ Upload a test PDF and cover image under `/admin/books/new` → confirm upload progress reaches `100%` and book saves.
    - ☐ Open `/library` in an incognito window → confirm the book appears, opens in the PDF.js reader (`/library/books/[id]/read`), and obeys the `Allow Download` toggle (`403` when disabled, downloads when enabled).

---

11. Website Usage Guide

Admin Portal (`/admin`)

1. **Admin Login (`/admin/login`):** Enter your authorized administrator email and password. (Public signup is disabled.)
2. **Dashboard Overview (`/admin`):** View Total Books, Total PDFs, Total E-Books, Total Categories, cloud storage usage, Recently Uploaded Books, and Recent Admin Activity. Click **Quick Upload** or **Upload New Book** to publish immediately.
3. **Manage Categories (`/admin/categories`):** Create subject categories before uploading books. Click **New Category**, enter name, description, and choose an icon. Edit any category or delete empty categories.
4. **Add & Upload Books (`/admin/books/new`):**
   - Fill in Title*, Author*, Subject*, Category*, Description*, and optional Publisher, Publication Year, ISBN, Language, and Page Count.
   - Toggle **Allow Student Downloads** (on = downloadable + online reading; off = online reading only) and **Feature on Library Homepage**.
   - Select a `.pdf` or `.epub` file and optionally a cover image (`.jpg`, `.png`, `.webp`); wait for the upload progress bars to complete, then click **Publish Book to Library**.
5. **Edit, Replace Files, Control Permissions & Delete (`/admin/books` & `/admin/books/[id]/edit`):**
   - Use search, category, and format filters on `/admin/books`.
   - Click the **Allowed / Read Only** pill or **Featured / Standard** pill inline to toggle student permissions instantly.
   - Click the edit icon to modify metadata, replace the uploaded PDF/EPUB or cover image, or remove the cover image.

- Click the trash icon (`super_admin` and `admin` only) to permanently delete the database record and cloud storage files.

6. **Manage Administrators (`/admin/team`):** `super_admin` can add `admin` or `editor` users, change roles, suspend/activate accounts, or remove admins.

Student Portal (`/library`)

1. **Homepage (`/library`):** View total available resources, search directly via the hero search bar, explore **Featured Books** and **Recently Added Books**, or click any card under **Browse by Category**.
2. **Search & Filter Catalog (`/library/browse`):** Search across titles, authors, subjects, descriptions, publishers, and categories; filter by Category, Subject, Format (`PDF` / `EPUB`), or `Downloadable Only` ; sort by Recently Uploaded, Title (A–Z), Most Read, or Oldest; and navigate pages.
3. **Book Details (`/library/books/[id]`):** Inspect full synopsis, publisher/year, ISBN, format, size, and access status (`Read Online` / `Download` or `Read-Only`).
4. **Read Online (`/library/books/[id]/read`):**
   - **PDF Reader:** Zoom in/out (`+` / `−`), Fit Width (`Fit`), jump to a specific page number, use `ArrowLeft` / `ArrowRight` keyboard navigation, toggle Fullscreen, and download (if enabled).
   - **EPUB Reader:** Open the **Chapters** Table of Contents drawer, step between sections, adjust text size (`A−` / `A+`), switch between `light` , `sepia` , and `night` themes, and toggle Fullscreen.

---

12. API and Backend Documentation

All backend endpoints live under `src/app/api/` (plus `src/app/auth/callback/route.ts`).

Public & Authentication Endpoints

Method	Endpoint	Auth Required	Query / Body Parameters	Success Response	Error Codes
`GET`	`/api/health`	None	—	`200` : `{ status, service, supabaseConfigured, supabaseAdminConfigured, timestamp }`	—
`GET`	`/api/books`	None	Query: `q` , `category` (slug/UUID), `subject` , `format` (`all` | `pdf` | `epub`), `downloadable` (`all` | `true`), `sort` (`newest` | `oldest` | `title_asc` | `most_viewed`), `page` , `limit` (`1..48`)	`200` : `{ configured, books, total, page, limit, totalPages, subjects }`	`503` (unconfigured)
`GET`	`/api/books/[id]`	None	Path: `id` (UUID)	`200` : `{ book }`	`400` , `404` , `503`
`GET` / `HEAD`	`/api/books/[id]/stream`	None	Path: `id` (UUID); optional HTTP `Range` header	`200` or `206` binary stream (`Content-Disposition: inline` , `Accept-Ranges: bytes`)	`400` , `404` , `502` , `503`
`GET`	`/api/books/[id]/download`	None	Path: `id` (UUID)	`302` redirect to 60s signed Supabase Storage download URL (and increments `download_count`)	`400` , `403` (download disabled), `404` , `502` , `503`
`POST`	`/api/books/[id]/view`	None	Path: `id` (UUID)	`200` : `{ ok: true }` (calls `increment_book_metric(..., 'view')`)	`400` , `503`
`GET`	`/api/categories`	None	—	`200` : `{ categories }` (with `book_count`)	`503`
`POST`	`/api/auth/logout`	Cookie session	—	`200` : `{ ok: true }`	—
`GET`	`/auth/callback`	None	Query: `code` , optional `next` (`/admin*`)	`302` redirect to `next` (or `/admin/login` on error)	—

Protected Admin Endpoints

All `/api/admin/*` routes require a valid Supabase Auth session **and** an active row in `public.admin_users` (`401` if unauthenticated, `403` if inactive or insufficient role, `503` if cloud config missing).

Method	Endpoint	Allowed Roles	Request Parameters / JSON Body	Success Response	Error Codes
`GET`	`/api/admin/overview`	Any active admin	—	`200` : `{ stats, recentBooks, recentLogs }`	`401` , `403` , `503`
`POST`	`/api/admin/uploads/sign`	Any active admin	`{ kind: 'book'|'cover', fileName, fileType, fileSize }`	`200` : `{ signedUrl, token, path, bucket, fileType, mimeType }`	`400` , `401` , `403` , `502` , `503`
`POST`	`/api/admin/uploads/cleanup`	Any active admin	`{ bucket: 'book-files'|'book-covers', path }`	`200` : `{ ok: true }`	`400` , `401` , `403` , `409` (attached to saved book), `503`
`GET`	`/api/admin/books`	Any active admin	Query: `q` , `categoryId` , `format`	`200` : `{ books }`	`401` , `403` , `503`
`POST`	`/api/admin/books`	Any active admin	`createBookSchema` : `title` , `author` , `description` , `subject` , `category_id` , `file_storage_path` , `file_name` , `file_type` , `mime_type` , `file_size` , optional metadata & cover fields, `allow_download` , `is_featured`	`201` : `{ book }` *(cleans up uploaded files if DB insert fails)*	`400` , `401` , `403` , `503`
`GET`	`/api/admin/books/[id]`	Any active admin	Path: `id` (UUID)	`200` : `{ book }`	`400` , `401` , `403` , `404` , `503`
`PATCH`	`/api/admin/books/[id]`	Any active admin	Partial `updateBookSchema` + optional `remove_cover: boolean`	`200` : `{ book }` *(removes old storage objects when replaced/removed)*	`400` , `401` , `403` , `404` , `503`
`DELETE`	`/api/admin/books/[id]`	`super_admin` , `admin`	Path: `id` (UUID)	`200` : `{ ok: true }` *(deletes DB row + files from both buckets)*	`400` , `401` , `403` , `404` , `503`
`GET`	`/api/admin/categories`	Any active admin	—	`200` : `{ categories }`	`401` , `403` , `503`
`POST`	`/api/admin/categories`	Any active admin	`{ category_name, description?, icon_name? }`	`201` : `{ category }`	`400` , `401` , `403` , `409` (duplicate), `503`
`PATCH`	`/api/admin/categories/[id]`	Any active admin	Path: `id` (UUID); `{ category_name, description?, icon_name? }`	`200` : `{ category }`	`400` , `401` , `403` , `409` , `503`
`DELETE`	`/api/admin/categories/[id]`	`super_admin` , `admin`	Path: `id` (UUID)	`200` : `{ ok: true }`	`400` , `401` , `403` , `409` (books assigned), `503`
`GET`	`/api/admin/team`	Any active admin	—	`200` : `{ admins, currentAdminId }`	`401` , `403` , `503`
`POST`	`/api/admin/team`	`super_admin` only	`{ name, email, role: 'admin'|'editor', mode: 'password'|'invite', password? }`	`201` : `{ admin }`	`400` , `401` , `403` , `409` , `503`
`PATCH`	`/api/admin/team/[id]`	`super_admin` only	Path: `id` (UUID); `{ role?, is_active?, name? }`	`200` : `{ admin }`	`400` , `401` , `403` , `404` , `503`
`DELETE`	`/api/admin/team/[id]`	`super_admin` only	Path: `id` (UUID)	`200` : `{ ok: true }` *(deletes from `admin_users` + `auth.users`)*	`400` , `401` , `403` , `404` , `503`

13. Troubleshooting Guide

Issue	Likely Cause	Resolution
Website does not start locally	Outdated Node.js version or missing dependencies.	Ensure `node -v` is `v20+`. Delete `node_modules` and run `npm install`.
`npm install` fails	Network proxy restriction or locked file.	Retry `npm install`; check npm registry access (`npm ping`) and ensure no stray process locks `.next` or `node_modules`.
`npm run build` fails	TypeScript or ESLint errors, or missing `pdf.worker.min.mjs`.	Run `npm run typecheck && npm run lint`. Ensure `node scripts/copy-pdf-worker.mjs` runs (automatically invoked by `npm run build`).
Supabase connection errors / SetupNotice shown	`.env.local` missing, restarted dev server needed, or placeholder keys still present.	Verify `.env.local` has real values (not `your-project-ref`), check `/api/health`, and restart `npm run dev`.
Invalid API keys error	Copied `Secret key` into `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` or mistyped URL/key.	Re-copy **Project URL**, **Publishable key**, and **Secret key** from **Supabase → Settings → API Keys**. Ensure URL starts with `https://`.
Admin login succeeds in Auth but says "not authorized"	Missing or inactive row in `public.admin_users`.	Run `npm run verify-setup`, check `public.admin_users` in Supabase Table Editor (`is_active = true`), or run `npm run bootstrap-admin`.
Database permission / relation does not exist errors	SQL migration was not executed on the active Supabase project.	Execute `supabase/migrations/202611100001_initial_schema.sql` in the Supabase SQL Editor, then run `npm run verify-setup`.
RLS policy issues on direct queries	Modifying tables directly using the browser `anon` /publishable key without admin login.	Admin writes must run while signed in as an active admin in `admin_users` or via server API routes using `SUPABASE_SECRET_KEY`.
PDF/EPUB upload fails	File exceeds `MAX_BOOK_FILE_SIZE_MB` (`50 MB`) / `100 MB` bucket limit, wrong extension, or missing buckets.	Verify file is `.pdf` or `.epub`, within size limits, and that `book-files` and `book-covers` buckets exist (`npm run verify-setup`).
Storage permission errors on upload/delete	`SUPABASE_SECRET_KEY` missing or storage bucket policies missing.	Verify `SUPABASE_SECRET_KEY` in `.env.local` /Vercel and re-run Section 16–17 of the migration SQL.
PDF viewer stays blank or shows worker error	Missing `/pdf.worker.min.mjs` in `public/`.	Run `node scripts/copy-pdf-worker.mjs`, verify `http://localhost:3000/pdf.worker.min.mjs` returns `200`, and check `/api/books/[id]/stream` in browser Network tab.
Vercel deployment fails	Missing environment variables or build mismatch.	Check Vercel Build Logs, confirm Node.js 20.x in Project Settings, and confirm `NEXT_PUBLIC_SUPABASE_URL` & keys are configured.
Invite/reset links redirect to `localhost:3000`	`NEXT_PUBLIC_SITE_URL` still set to `http://localhost:3000` in Vercel or Supabase Site URL not updated.	Update `NEXT_PUBLIC_SITE_URL` in Vercel and **Site URL / Redirect URLs** in Supabase Auth settings.

14. Maintenance, Backup and Updates

- **Updating the Application:** Pull or commit code updates, run `npm run typecheck && npm run lint && npm test && npm run build`, and push to `main` (Vercel auto-deploys).
- **Installing / Updating Dependencies:** Run `npm install <package>`, inspect `npm audit`, and verify `postinstall` still copies `public/pdf.worker.min.mjs` whenever `pdfjs-dist` updates.
- **Database Backups:**
  - **Automated:** Enable automated daily backups (or Point-in-Time Recovery on Supabase Pro) in **Supabase Dashboard → Database → Backups**.
  - **Manual Export:** Periodically run `pg_dump "$DATABASE_URL" -Fc -f backup-$(date +%F).dump` from a secure administrative workstation using the PostgreSQL URI from Supabase Settings.
- **Cloud File Backups (PDFs, EPUBs, Covers):** Sync/export the `book-files` and `book-covers` buckets periodically via Supabase Storage S3 compatibility or CLI scripts to institutional archival storage.
- **Restoring:** Restore PostgreSQL dumps via `pg_restore` into a clean Supabase instance, re-upload backed-up objects to `book-files` and `book-covers` preserving the exact `storage_path` strings stored in `books.file_storage_path` and `books.cover_image_path`.
- **Monitoring Storage & Activity:** Check the **Cloud Storage Health** and **Recent Admin Activity** cards on `/admin` alongside **Supabase Dashboard → Reports / Storage**.

15. Security Best Practices

- **Protect Secret Credentials:** Keep `SUPABASE_SECRET_KEY` and `DATABASE_URL` strictly in server-side environment variables (`.env.local` and Vercel encrypted env vars). Never expose them in client code or Git history.
- **Keep `.env*` Out of Git:** Confirm `.gitignore` continues ignoring `.env` and `.env*.local` before every commit.
- **Enforce Database RLS:** Keep Row Level Security enabled on all 5 tables and both storage buckets; never disable RLS in production.
- **Maintain Least-Privilege Roles:** Grant `super_admin` only to the Head/Primary Library Administrator; assign `admin` or `editor` to departmental staff. Immediately suspend (`is_active = false`) or remove departing staff from `/admin/team`.
- **Strict File & Input Validation:** All inputs pass Zod schemas (`src/lib/validation.ts`), search terms strip PostgREST filter syntax (`sanitizeSearchTerm`), storage paths reject `..` traversal, and uploads validate extensions, MIME types, and byte limits on both client and server.
- **HTTP Security Headers:** `next.config.ts` automatically sets `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: strict-origin-when-cross-origin`, and `Permissions-Policy: camera=(), microphone=(), geolocation=()`.
- **Routine Dependency Patching:** Run `npm outdated` and `npm audit` periodically and re-run `npm test` + `npm run build` after any update.